

Q: A.2 – Learning rate, batch size, initialization (2 pts)

You now have:

- **Chosen hyper-parameters**

- Architecture: MLP with layers **2-10-10-2**, ReLU in hidden layers, softmax at output
- Loss: **cross-entropy**
- **Learning rate:** $\eta = 0.1$
- **Batch size:** $B = 16$ (mini-batch SGD)
- **Weight initialisation:** $w \sim N(0, 0.12)$, $b \sim N(0, 0.12)$, biases = 0
- **Pre-processing:** standardisation of X_0, X_1 using training mean/std

- **Justification evidence**

- LR sweep plots for $\eta \in \{0.1, 0.01, 0.001\}$:
 - 0.01 and 0.001 $\rightarrow$ almost flat loss (too small LR, barely learning)
 - 0.1 $\rightarrow$ clear decrease in loss (only one that learns properly with full batch)
- Batch size sweep plots for $B \in \{\text{full}, 16, 4\}$ (with $\text{lr}=0.1, \sigma=0.1$):
 - full batch $\rightarrow$ slow but stable, higher final loss
 - 4 $\rightarrow$ noisy, validation loss oscillates and trends upward
 - 16 $\rightarrow$ fast drop, low final loss, small noise $\rightarrow$ best trade-off
- Init sweep plots for $\sigma \in \{0.01, 0.1, 1.0\}$ (with $\text{lr}=0.1, B=16$):
 - 0.01 $\rightarrow$ long plateau at ~ 0.7 then late drop (gradients too small)
 - 1.0 $\rightarrow$ more spikes and instability in validation loss
 - 0.1 $\rightarrow$ smooth fast convergence, low and stable val loss

As long as you **describe this reasoning in the report and show a few of those plots**, Q2 is clearly satisfied.

Q: A.3 – Training, plots, stopping criterion, final metrics (2 pts)

You have:

- **Plots** of training vs validation loss for:
 - Different learning rates
 - Different batch sizes
 - Different init scales
 - **Final model** ($lr=0.1$, $B=16$, $\sigma=0.1$)
- **Stopping criterion (what to write)**
 - Loss for the final model drops sharply during the first ~20 epochs, then plateaus around **0.1 (train)** and **0.13–0.15 (val)**.
 - After ~150–200 epochs, the validation curve is basically flat with only small noise.
 - The gap between train and val loss is small and stable → mild overfitting only.
 - So you can say:

"We trained for 200 epochs; beyond this point the validation loss stopped improving significantly, so we consider the model to have trained sufficiently."
- **Final metrics on validation set**
 - Final validation loss: **0.1292**
 - Final validation accuracy: **0.9756** (80 / 82 correct)
 - Confusion matrix (rows = true, cols = predicted):
[40 1
1 40]

```

=== THA2: MLP Training (Part A.3) ===
Training samples: 410
Validation samples: 82

=== Experiment: lr=0.1, batch_size=None, init_scale=0.1 ===
Final Val Loss: 0.6589, Final Val Acc: 0.8293

=== Experiment: lr=0.01, batch_size=None, init_scale=0.1 ===
Final Val Loss: 0.6931, Final Val Acc: 0.6707

=== Experiment: lr=0.001, batch_size=None, init_scale=0.1 ===
Final Val Loss: 0.6923, Final Val Acc: 0.8659

[SWEEP] Best learning rate (by val loss): 0.1

=== Experiment: lr=0.1, batch_size=None, init_scale=0.1 ===
Final Val Loss: 0.6012, Final Val Acc: 0.6951

=== Experiment: lr=0.1, batch_size=16, init_scale=0.1 ===
Final Val Loss: 0.1727, Final Val Acc: 0.9512

=== Experiment: lr=0.1, batch_size=4, init_scale=0.1 ===
Final Val Loss: 0.2007, Final Val Acc: 0.9756
[SWEEP] Best batch size (by val loss): 16

=== Experiment: lr=0.1, batch_size=16, init_scale=0.01 ===
Final Val Loss: 0.1271, Final Val Acc: 0.9756

=== Experiment: lr=0.1, batch_size=16, init_scale=0.1 ===
Final Val Loss: 0.1177, Final Val Acc: 0.9756

=== Experiment: lr=0.1, batch_size=16, init_scale=1.0 ===
Final Val Loss: 0.1511, Final Val Acc: 0.9756
[SWEEP] Best init scale (by val loss): 0.1

=== Final training with lr=0.1, batch_size=16, init_scale=0.1 ===

```

```

=== Final training with lr=0.1, batch_size=16, init_scale=0.1 ===
=====
MLP Network Architecture
=====
Input Layer (x): 2 neurons
Hidden Layer 0 (h(0)): 10 neurons
Hidden Layer 1 (h(1)): 10 neurons
Output Layer (o): 2 neurons
=====
Activation Function: relu
Learning Rate: 0.1
Init Scale ( $\sigma$ ): 0.1
=====

Weight Matrices:
W0: (2, 10)
W1: (10, 10)
W2: (10, 2)

Bias Vectors:
b0: (1, 10)
b1: (1, 10)
b2: (1, 2)
=====
Epoch 001 | Train Loss: 0.6942 | Val Loss: 0.6921 | Val Acc: 0.5488
Epoch 010 | Train Loss: 0.6415 | Val Loss: 0.6241 | Val Acc: 0.7073
Epoch 020 | Train Loss: 0.2274 | Val Loss: 0.2009 | Val Acc: 0.9634
Epoch 030 | Train Loss: 0.1375 | Val Loss: 0.1537 | Val Acc: 0.9756
Epoch 040 | Train Loss: 0.1183 | Val Loss: 0.1464 | Val Acc: 0.9756
Epoch 050 | Train Loss: 0.1138 | Val Loss: 0.1510 | Val Acc: 0.9756
Epoch 060 | Train Loss: 0.1105 | Val Loss: 0.1454 | Val Acc: 0.9634
Epoch 070 | Train Loss: 0.1103 | Val Loss: 0.1441 | Val Acc: 0.9756
Epoch 080 | Train Loss: 0.1062 | Val Loss: 0.1432 | Val Acc: 0.9756
Epoch 090 | Train Loss: 0.1037 | Val Loss: 0.1432 | Val Acc: 0.9756
Epoch 100 | Train Loss: 0.1032 | Val Loss: 0.1488 | Val Acc: 0.9756
Epoch 110 | Train Loss: 0.1075 | Val Loss: 0.1436 | Val Acc: 0.9756
Epoch 120 | Train Loss: 0.1003 | Val Loss: 0.1407 | Val Acc: 0.9756
Epoch 130 | Train Loss: 0.0993 | Val Loss: 0.1443 | Val Acc: 0.9756
Epoch 140 | Train Loss: 0.0973 | Val Loss: 0.1480 | Val Acc: 0.9756
Epoch 150 | Train Loss: 0.1016 | Val Loss: 0.1407 | Val Acc: 0.9756
Epoch 160 | Train Loss: 0.0992 | Val Loss: 0.1384 | Val Acc: 0.9756
Epoch 170 | Train Loss: 0.0943 | Val Loss: 0.1425 | Val Acc: 0.9756
Epoch 180 | Train Loss: 0.1008 | Val Loss: 0.1371 | Val Acc: 0.9756
Epoch 190 | Train Loss: 0.0979 | Val Loss: 0.1385 | Val Acc: 0.9756
Epoch 200 | Train Loss: 0.0985 | Val Loss: 0.1451 | Val Acc: 0.9756

--- Final Evaluation on Validation Set ---
Final Validation Loss: 0.1451
Final Validation Accuracy: 0.9756
Confusion Matrix (rows = true, cols = pred):
[[40  1]
 [ 1 40]]

```

1. LR sweep – **lr = 0.1**, batch = full, init = 0.1

Plot title: *Train vs Val Loss (lr=0.1, batch=full, init=0.1)*

- **What it is:** Training and validation loss vs epoch for a relatively large learning rate with full-batch gradient descent.
- **What it shows:** Both losses decrease slowly from ~0.693 to ~0.659 over 200 epochs. Training and validation curves almost overlap → stable but **very slow learning**. This is better than random, but still quite high loss.

2. LR sweep – **lr = 0.01**, batch = full, init = 0.1

Plot title: ... (*lr=0.01, batch=full, init=0.1*)

- **What it is:** Same setup as above but with a smaller learning rate.
- **What it shows:** Loss decreases even more slowly and only by a tiny amount (~0.6947 → ~0.6931). The model basically doesn't learn in 200 epochs – **too small learning rate**.

3. LR sweep – **lr = 0.001**, batch = full, init = 0.1

Plot title: ... (*lr=0.001, batch=full, init=0.1*)

- **What it is:** Even smaller learning rate.
- **What it shows:** Loss curves are almost flat, with a microscopic decrease. This confirms **lr = 0.001** is **far too small** in this configuration.

*(From these three, we conclude that among full-batch runs, **lr = 0.1** is the only one that makes reasonable progress.)*

4. Batch-size sweep – **bs = full**, lr = 0.1, init = 0.1

Plot title: ... (*lr=0.1, batch=full, init=0.1*)

- **What it is:** Baseline for the batch-size experiment (same as Fig. 1).
- **What it shows:** Smooth convergence but fairly slow, with loss around 0.60–0.66 at the end. This is the **slow but stable full-batch reference**.

5. Batch-size sweep – **bs = 16**, lr = 0.1, init = 0.1

Plot title: ... (*lr=0.1, batch=16, init=0.1*)

- **What it is:** Mini-batch SGD with batch size 16.
- **What it shows:** Loss plunges quickly from ~ 0.69 to ~ 0.16 in ~ 20 epochs, then oscillates around ~ 0.13 – 0.15 . Validation curve follows train curve with small noise. This is **fast convergence and good generalisation**.

6. Batch-size sweep – **bs = 4**, lr = 0.1, init = 0.1

Plot title: ... (lr=0.1, batch=4, init=0.1)

- **What it is:** Very small mini-batch size.
- **What it shows:** Training loss is low and smooth, but the validation loss is **much noisier** with frequent spikes and an upward trend. This indicates **too much stochastic noise** and reduced stability.
- **Conclusion:** **bs = 16** is a better trade-off than 4 or full batch.

7. Init sweep – **init_scale = 0.01**, lr = 0.1, bs = 16

Plot title: ... (lr=0.1, batch=16, init=0.01)

- **What it is:** Same LR and batch size, but smaller weight standard deviation.
- **What it shows:** For many epochs (~ 0 – 120) loss stays stuck near 0.70, then suddenly drops sharply and stabilises around ~ 0.12 – 0.15 . This “delayed learning” suggests **initial activations/gradients are too small**, so training starts effectively very late.

8. Init sweep – **init_scale = 1.0**, lr = 0.1, bs = 16

Plot title: ... (lr=0.1, batch=16, init=1.0)

- **What it is:** Much larger initial weights.
- **What it shows:** Loss drops quickly but the validation curve is **quite noisy with frequent spikes**. Large weights make the network more unstable and sensitive to mini-batch noise. Generalisation is worse than with $\sigma = 0.1$.

9. Init sweep – **init_scale = 0.1**, lr = 0.1, bs = 16

Plot title: ... (lr=0.1, batch=16, init=0.1)

- **What it is:** Intermediate initialisation scale (the one we finally chose).

- **What it shows:** Loss drops rapidly in the first ~20 epochs and then stabilises. Both curves are low and relatively smooth, with only small validation noise. This gives **fast, stable convergence and low final loss**, so $\sigma = 0.1$ is selected.

10. Final model – `lr = 0.1` , `bs = 16` , `init_scale = 0.1`

Plot title: *Training vs Validation Loss (final model)*

- **What it is:** Training of the final chosen configuration, for 200 epochs.
- **What it shows:**
 - Early phase: sharp loss drop from ~0.69 to ~0.17 in ~20 epochs.
 - Later phase: gradual decrease toward ~0.10 (train) and ~0.13–0.14 (val).
 - The gap between curves is small and roughly constant → **good generalisation with mild overfitting only**.

Q: B.1 Zero-initialised model – behaviour

- In this experiment we kept the same architecture and hyper parameters as the best model from Part A
(MLP 2-10-10-2, ReLU, `lr = 0.1`, batch size = 16, standardised inputs)
but set all weights and biases to zero before training.
- The plot shows training and validation loss remaining almost flat around 0.693 (which is $\log_2 \log_2$, the loss of a random 50/50 classifier) during all 200 epochs.
- The training loss curve is noisy, because it is computed on mini-batches; it oscillates slightly above and below 0.693 with no clear downward trend.
- The validation loss curve is lower-variance and almost constant, again close to 0.693, indicating that the model does not systematically improve on the validation set.
- The corresponding validation accuracy is close to 50% (insert your exact number from the log) and the confusion matrix is roughly balanced between the two classes, i.e. the network behaves essentially like random guessing.

With all parameters initialised to zero, all neurons in a given layer start with exactly the same weights and biases. Since they receive the same inputs, they produce identical outputs and identical gradients (symmetry). As a consequence, the hidden units remain indistinguishable throughout training and the network effectively behaves like a very limited linear classifier. Although stochastic gradient descent perturbs the parameters slightly, the loss stays around $\log 2 \approx 0.693$ and the model never learns a meaningful decision boundary.

In contrast, the randomly initialised model from Part A (weights $\sim N(0, 0.1^2)$) quickly reduced the loss to ≈ 0.13 and achieved $\approx 97.6\%$ validation accuracy, with a confusion matrix showing only 2 misclassified samples. This comparison illustrates why **proper random initialisation is crucial** for deep networks: zero initialisation leads to symmetry and prevents the MLP from learning useful representations.

Q: B.2 Description of the heatmap

For this experiment we fixed the MLP architecture and all training settings from Part A (2–10–10–2, ReLU, batch size 16, 100 epochs, standardised inputs) and varied:

- **Learning rate** $\eta \in \{10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}\} \rightarrow \mathbf{y\text{-axis}}$
- **Initialisation variance** $\sigma^2 \in \{0, 0.1, \dots, 1.0\}$ for weights and biases drawn from $N(0, \sigma^2) \rightarrow \mathbf{x\text{-axis}}$

Each cell in the heatmap shows the **validation accuracy** after training once with that (η, σ^2) .

Main observations

You can write something along these lines:

1. **Zero variance ($\sigma^2 = 0$) always gives 50% accuracy.**

The first column is consistently dark (0.5) for all learning rates.

This corresponds to all parameters initialised to zero: the network collapses to predicting a constant class and never learns a useful decision boundary, exactly as in B.1.

2. **Very small learning rate ($\eta = 10^{-5}$) underfits for all σ^2 .**

Accuracies are mostly between 0.32 and 0.63, never close to the ~ 0.97 we know is achievable.

The optimiser makes very small updates, so the effect of initialisation dominates and the model cannot properly fit the data.

3. Small learning rate ($\eta = 10^{-4}$) shows some sensitivity to σ^2 .

- For moderate σ^2 (0.4–0.6, 0.9–1.0) accuracy rises to ≈ 0.60 –0.83.
- For other σ^2 it stays much lower (≈ 0.34 –0.66).

This shows that appropriate initialisation can partially compensate for a small learning rate, but not fully.

4. Intermediate learning rate ($\eta = 10^{-3}$) + moderate σ^2 gives very high accuracy.

- For σ^2 between about 0.1 and 1.0, validation accuracy is typically ≥ 0.89 , peaking at 0.9756 for several σ^2 values (0.3, 0.7, 0.9, 1.0).
- Only $\sigma^2 = 0$ remains at 0.5.

This region shows a good trade-off: gradients are large enough and initial weights are in a scale where ReLUs are active but not saturated.

5. Large learning rates ($\eta = 10^{-2}$ and 10^{-1}) work extremely well as soon as $\sigma^2 > 0$.

For both rows:

- $\sigma^2 = 0 \rightarrow 0.5$ accuracy (again, the degenerate zero-init case).
- For all $\sigma^2 \geq 0.1$ the accuracy jumps to 0.9756 and stays there.

This means that once we break symmetry ($\sigma^2 > 0$), a relatively large learning rate lets the network quickly converge to essentially the same near-optimal solution, and additional changes in σ^2 no longer matter much.

Q: B.3 Best model ($\eta = 1e-2$, $\sigma^2 = 0.1$, val acc ≈ 0.9756)

1. best_b3 – hidden layer 1 (init)

- Activations are random and low-structured: both classes (top and bottom halves of the y-axis) look very similar.

- A few neurons already have slightly higher activations, but there is no clear separation between classes.
- Interpretation: with random Gaussian init, the first hidden layer represents random projections of the input, not yet class-specific.

2. **best_b3 – hidden layer 2 (init)**

- Same story at a deeper layer: random speckled pattern, no obvious blocks by class.
- Some neurons (e.g. near the right side) show larger activations, but they respond similarly to both classes.
- Interpretation: initially, the second hidden layer just composes random features from layer 1.

3. **best_b3 – hidden layer 1 (mid)**

- Compared to init, you now see more structure:
 - Some neurons are more active on one half of the samples than the other (different colour intensities for class-0 vs class-1 bands).
 - Activations are still fairly diverse across neurons → no dead neurons.
- Interpretation: midway through training the first layer has learned class-dependent features, starting to separate the two classes in feature space.

4. **best_b3 – hidden layer 2 (mid)**

- Much more blocky and high-contrast than at init:
 - Certain neurons fire strongly almost only for one class, while being near zero for the other.
 - A few neurons dominate with higher activation ranges (colour bar goes higher than layer 1).
- Interpretation: second layer is learning high-level, class-specific combinations of layer-1 features, which is exactly what we want in a good classifier.

5. **best_b3 – hidden layer 1 (final)**

- Pattern is similar to the mid-epoch plot but slightly sharpened:
 - Class bands are clearer; some neurons now consistently activate strongly only for one class.
 - Activation values are moderate (no explosion) and most neurons remain useful.
- Interpretation: the first layer has converged to a stable, well-scaled representation that still preserves diversity across neurons.

6. **best_b3 – hidden layer 2 (final)**

- Very clear class-separation:
 - Strong vertical bands for particular neurons that activate much more on one class than the other.
 - Some neurons are almost “on/off” by class, reflecting near-binary decisions.
- Interpretation: this layer implements a discriminative representation, mapping the two classes to clearly separated activation patterns, which is consistent with the high final validation accuracy ($\approx 97.6\%$).

Worst model ($\eta = 1e-5$, $\sigma^2 = 1.0$, val acc ≈ 0.45)

7. **worst_b3 – hidden layer 1 (init)**

- Activations are much larger (colour bar up to around 6) and less controlled.
- Many neurons have strong activations across *both* classes (bright columns), so they’re not selective at all.
- Interpretation: big initial weights ($\sigma^2 = 1.0$) push ReLU activations into a high-magnitude regime, but without any class structure.

8. **worst_b3 – hidden layer 1 (mid)**

- The pattern looks almost identical to the init plot:

- No obvious change in which neurons are active or how they react to each class.
- No emerging class-specific patterns.
- Interpretation: with such a tiny learning rate ($1e-5$), the network barely moves away from its random initialisation; parameters change so slowly that the representation is effectively frozen.

9. **worst_b3 – hidden layer 2 (init)**

- Even more extreme: activations go up to ~ 20 .
- Several neurons saturate with very large values for most samples, regardless of class.
- Interpretation: the second layer is already in a highly unbalanced and saturated regime at $t = 0$, making optimisation difficult.

10. **worst_b3 – hidden layer 2 (mid)**

- Again, almost no visible evolution relative to init; the bright columns and general pattern are basically the same.
- No clear separation between top and bottom halves (classes) $\rightarrow$ poor class-specific encoding.
- Interpretation: combined with the tiny learning rate, these large initial weights lead to a network that doesn't significantly adapt; it ends up behaving like a near-constant function, which matches the low accuracy ($\sim 45\%$).

11. **worst_b3 – hidden layer 1 (final)**

- What it shows: ReLU activations of the 10 neurons in hidden layer 1, at the end of training for the worst model ($\eta = 1e-5$, $\sigma^2 = 1.0$).
- Pattern:
 - The picture looks *almost the same* as at initialisation and mid-training.
 - Activations are still quite large for many neurons (colour bar up to ~ 6) and spread across both classes.

- There is no clear difference between the top half and bottom half of the samples (class 0 vs class 1).
- Interpretation:
The very small learning rate means weights barely change, so the first hidden layer remains close to a random, badly-scaled projection of the input. It does not learn class-specific features, which fits the poor accuracy (~45–50%).

12. **worst_b3 – hidden layer 2 (final)**

- What it shows: ReLU activations of the 10 neurons in hidden layer 2 at the end of training for the same worst model.
- Pattern:
 - Again, the pattern is almost identical to the init/mid plots, but with very large activations (colour bar up to ~20).
 - Several neurons are saturated with high values for almost *all* samples, regardless of class.
 - There is no clean separation between classes along the y-axis.
- Interpretation:
The second hidden layer is stuck in a highly saturated regime and never manages to carve out decision-boundaries between the two classes. Combined with the tiny learning rate, the network effectively behaves like a near-constant high-activation mapping and ends up with chance-level performance.